

题目 1

打开文件表由操作系统维护，存储当前已经通过 `open()` 系统调用打开且未关闭的文件属性条目。由于在文件系统中找到特定文件需要逐级目录深入访问，每次访问都会带来外存的文件定位开销，因此维护一个打开文件表可以使得同一个文件的后续访问可以**直接定位到其在外存上的位置**，显著减小文件访问的时间开销。

RAID 5：相比 RAID 4（使用一个单独的校验盘），**在每个磁盘上都放置一个校验条带**，从而把校验条带的读写开销均摊到每个磁盘上

RAID 6：采用两种校验算法计算出**两个独立的校验码**，每个磁盘上放置两个不同的校验条带

“755”文件权限：**所有者**可读写、可执行，文件所在**用户组**可读、可执行，**所有用户**可读、可执行

题目 2

采用 **FCFS** 算法：

当前处理的请求柱面号	当前请求磁头移动距离
2150	0
2069	81
1212	857
2296	1084
2800	504
544	2256
1618	1074
356	1262
1523	1167
4965	3442
3681	1284
总计移动距离	13011

采用 **SSTF** 算法：

当前处理的请求柱面号	当前请求磁头移动距离
2150	0
2069	81
2296	227
2800	504
3681	881
4965	1284
1618	3347
1523	95
1212	311
544	668
356	188
总计移动距离	7586

采用 **SCAN** 算法：

当前处理的请求柱面号	当前请求磁头移动距离
2150	0
2296	146
2800	504
3681	881
4965	1284
4999	34
2069	2930
1618	451
1523	95

当前处理的请求柱面号	当前请求磁头移动距离
1212	311
544	668
356	188
总计移动距离	7492

采用 **LOOK** 算法：

当前处理的请求柱面号	当前请求磁头移动距离
2150	0
2296	146
2800	504
3681	881
4965	1284
2069	2896
1618	451
1523	95
1212	311
544	668
356	188
总计移动距离	7424

2069, 1212, 2296, 2800, 544, 1618, 356, 1523, 4965, 3681

采用 **C-SCAN** 算法：

当前处理的请求柱面号	当前请求磁头移动距离
2150	0
2296	146

当前处理的请求柱面号	当前请求磁头移动距离
2800	504
3681	881
4965	1284
4999	34
0	4999
356	356
544	188
1212	668
1523	311
1618	95
2069	451
总计移动距离	9917

采用 **C-LOOK** 算法：

当前处理的请求柱面号	当前请求磁头移动距离
2150	0
2296	146
2800	504
3681	881
4965	1284
356	4609
544	188
1212	668
1523	311

当前处理的请求柱面号	当前请求磁头移动距离
1618	95
2069	451
总计移动距离	9137

题目 3

1.

- FAT16 文件系统目录项中文件大小使用 4 字节保存，该文件系统支持的文件字节长度上限为 $2^{4 \times 8} = 2^{32}$ 字节
- 又考虑到 FAT16 文件系统最多管理 2^{16} 个簇，每个簇的大小为 512B，因此该文件系统支持最大文件字节长度为 $2^{16+9} = 2^{25}$ 字节

2.

data 目录项位于根目录的第 2 个簇，file1.txt 目录项位于 data 目录的第 3 个簇。

不将读入的块缓存到内存中时，

- 在根目录中找到 data 目录项：顺序读取 2 个簇，触发 2 次磁盘 I/O
- 在 data 目录中找到 file1.txt 目录项：触发 5 次磁盘 I/O
 - 读 data 目录文件的第 1 个簇—查 FAT 表—读 data 目录文件的第 2 个簇—查 FAT 表—读 data 目录文件的第 3 个簇
- 读取 file1.txt 的前 1KB 数据：触发 3 次磁盘 I/O
 - 读 file1.txt 文件的第 1 个簇—查 FAT 表—读 file1.txt 文件的第 2 个簇
- 总计需要触发 **10 次**按块读取的磁盘 I/O

采用块缓存，并假设所有对 FAT 表的读取都在同一个块内时，

- 相较于不执行块缓存的情形，减少了两次从磁盘中读取 FAT 表的开销
- 总计需要触发 **8 次**按块读取的磁盘 I/O

3.

簇链设计的优劣势

- 优势：在扩张文件大小时更灵活，开销更少
 - 假设某个位于簇链中部的文件需要扩张大小，只需要在 FAT 表中**将属于该文件的末尾簇的后继指针指向簇链中的某个空闲簇**即可，而不需要将该文件的簇以后的所有簇向后移动以追求每个

文件始终占据磁盘上一段连续的簇

- 劣势：随机访问的性能较差
 - 在访问一个文件的后续簇时，需要依赖 FAT 表进行查找，可能因此带来若干次**额外的磁盘 I/O 开销**

簇链设计的最大优势在于保证文件大小动态增长的灵活性，而为此牺牲了文件读取时的性能；对于**只读存储介质**，连续分配情形下的文件读取表现将比簇链设计更好，同时节约了 FAT 表的存储开销。

题目 4

1.

- 14 个直接块，可存储 $512 \times 14 = 7168$ 字节数据
- 一个块指针大小为 4B，因此一个间接块中可以存储 $\frac{512}{4} = 128$ 个块指针
- 一级间接块可存储 $512 \times 128 = 65536$ 字节数据
- 因此整个文件系统支持的文件最大字节长度为 $7168 + 65536 = 72704$ 。

2.

- 文件原大小为 6KB，占用 12 个直接块
- 由于直接块数量为 14 个，因此新增的 3KB 文件大小需要占用 2 个直接块和一级间接块指向的 4 个数据块
- 总计需要分配 $2 + 1 + 4 = 7$ 个空闲数据块，其中 6 个数据块用于写入追加的文件数据，1 个数据块作为一级间接块写入指向 4 个已占用一级间接数据块的指针信息。

3.

- 当日志数据块已经写入，但日志头尚未写入时发生崩溃：内核恢复后未检测到有效日志头，因此**不会将位于日志区的新数据持久化入磁盘**
- 当内核已经更新了部分块时发生崩溃：内核恢复后检测到有效日志头，因此会**重新将日志区中的数据块按照日志头中保存的信息持久化入磁盘的对应位置**

题目 5

1.

- name 的前 8 个字节用于存储文件名，后 3 个字符用于存储扩展名
- first_cluster 用于存储该目录项对应的普通/目录文件在磁盘中的第一个数据簇簇号，并在必要时以此为根据**查 FAT 表获得后继簇号**
- 在 FAT16 文件系统下，最多管理 2^{16} 个簇，因此 first_cluster_hi 悬空，first_cluster 直接由 first_cluster_lo 得出，即 **first_cluster = first_cluster_lo**

2.

- cmd 0 命令执行完成后，磁盘上连续逻辑结构为 [KW0-A][DIRENT-A][DIRENT-B]
- cmd 1 命令执行完成后，磁盘上连续逻辑结构为
[KW0-A][DIRENT-A][KW0-B][KW1-B][KW2-B][KW3-B][KW4-B][KW5-B][DIRENT-B][DIRENT-C]
- cmd 2 命令执行完成后，磁盘上连续逻辑结构为
[KW0-A][DIRENT-A][DEL][DEL][DEL][DEL][KW0-B][KW1-B][DIRENT-B][DIRENT-C]
- cmd 3 命令执行完成后，磁盘上连续逻辑结构为
[KW0-A][KW1-A][KW2-A][KW3-A][DIRENT-A][DEL][KW0-B][KW1-B][DIRENT-B][DIRENT-C]